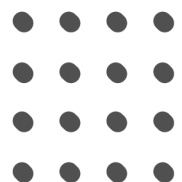




VMES Grinds

Week 4 - Heap





?

Theory

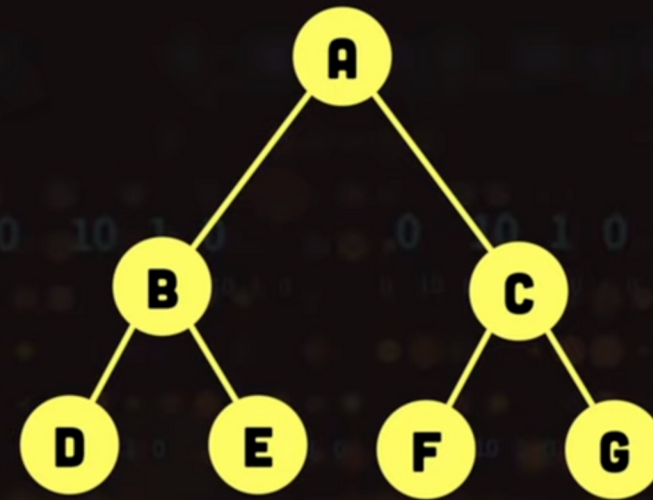


- **Heap is a tree based data structure.**
- **Complete binary tree**
- **one of the two types**
 - min heap
 - max heap



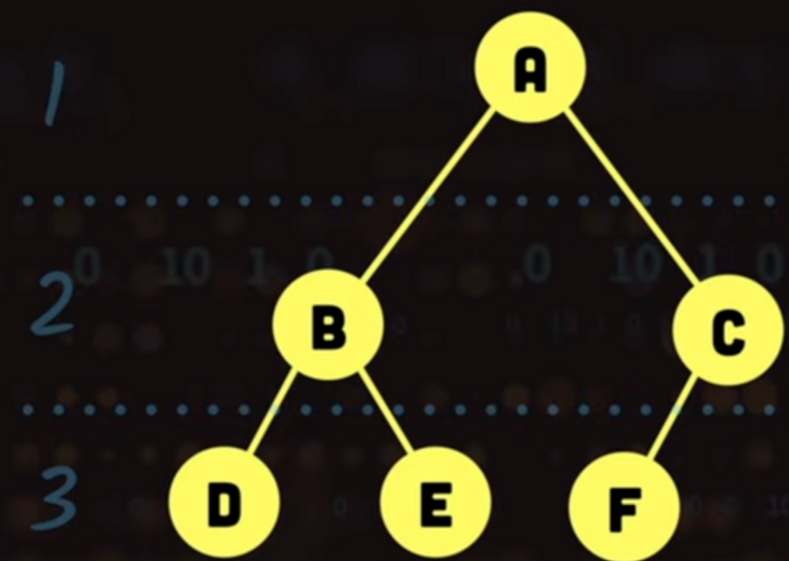
Full Binary Tree

All nodes of the binary tree
have 0 children or 2 children



Complete Binary Tree

All levels of the tree are filled
except last level which may be
filled from the left.





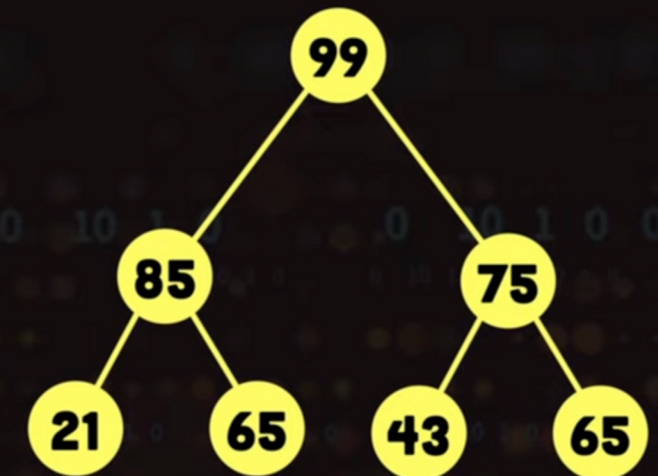
MIN HEAP

$value(parent) \leq value(child)$



MAX HEAP

$value(parent) \geq value(child)$





Min Heap Simulator: https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Min_Heap.html

Max Heap Simulator: https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Heap.html

Ref: <https://youtu.be/HqPJF2L5h9U?si=8Fqmk7ve0SY0L63c>

insert, delete

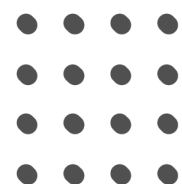


	Time complexity
heapify	$O(n)$
insert	$O(\log n)$
delete	$O(\log n)$
peek	$O(1)$



Pilot Program - Week 4

Heap





?

Leetcode: 1046. Last Stone Weight



BHONE



1046. Last Stone Weight

Solved

Easy

Topics

Companies

Hint

You are given an array of integers `stones` where `stones[i]` is the weight of the i^{th} stone.

We are playing a game with the stones. On each turn, we choose the **heaviest two stones** and smash them together. Suppose the heaviest two stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is **at most one** stone left.

Return *the weight of the last remaining stone*. If there are no stones left, return `0`.



Example 1:

Input: stones = [2,7,4,1,8,1]

Output: 1

Explanation:

We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,
we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,
we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the last stone.

Example 2:

Input: stones = [1]

Output: 1



Constraints:

- `1 <= stones.length <= 30`
- `1 <= stones[i] <= 1000`



Explanation





VMES Grinds



Code and Optimised Solution





```
1 import heapq
2
3 class Solution:
4     def lastStoneWeight(self, stones: List[int]) -> int:
5         if len(stones) == 1:
6             return stones[0]
7         stones = [-stone for stone in stones]
8         heapq.heapify(stones)
9
10        while len(stones) > 1:
11            first = heapq.heappop(stones) * -1
12            second = heapq.heappop(stones) * -1
13
14            if first != second:
15                heapq.heappush(stones, (first - second) * -1)
16        return stones[0] * -1 if stones else 0
```



Time and Space Complexity Analysis





```
1 import heapq
2
3 class Solution:
4     def lastStoneWeight(self, stones: List[int]) -> int:
5         if len(stones) == 1:
6             return stones[0]
7         stones = [-stone for stone in stones]
8         heapq.heapify(stones)
9
10        while len(stones) > 1:
11            first = heapq.heappop(stones) * -1
12            second = heapq.heappop(stones) * -1
13
14            if first != second:
15                heapq.heappush(stones, (first - second) * -1)
16        return stones[0] * -1 if stones else 0
```

} Heapify takes $O(n)$

} $O(n \log n)$

Time - $O(n \log n)$
Space - $O(1)$



Key Takeaway



- # Takeaway 1: When you need to repeatedly access/remove the largest/smallest elements, think max, or min Heap
- # Takeaway 2: Python heapq Workaround